

Assignment 1 Report

Student name: Wenxing Du

Student ID: 30171857

Student's Monash email: wduu0005@student.monash.edu

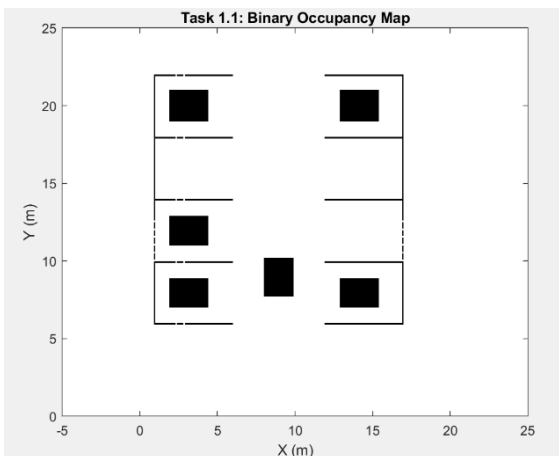
Submission date: 02/05/2025

1. Part 1

1.1. Binary occupancy map

I create a binary occupancy grid map based on the parking lot layout shown in the parkinglot.m file. This map is used to represent obstacles and traversable areas in the environment and will be used for path planning later.

First, use the binaryOccupancyMap class to initialize an empty map. The X and Y range of the map is 0~25 meters. The



resolution is set to 10, which means that each meter is divided into 10 grids, that is, each grid is 0.1 meters. Secondly, use the setOccupancy() function to manually set the parking space lines and boundary lines on the left and right sides of the parking lot to occupied status (value is 1) section by section. Based on the vehicle size given in the question (2.4m × 1.8m), I consider the 6 static vehicles in the parking lot as rectangular obstacles. Call the custom function markRectangle() to mark the location area of these vehicles as occupied. The step size is set to 0.05 meters to cover as many obscured grid points as possible at a resolution of 0.1 meters. Finally, I use show(map) to visualize the output results, and use the save () function to save it as a .mat file for subsequent tasks.

Figure 1: Binary occupancy map

1.2. Planning using RRT*

Firstly, I use load () to load the original map saved in Task 1.1. Then I use inflate () to inflate the map by 1.1 meters, making sure to account for the width of the vehicle and the safety margin.

Secondly, create a state space (with X, Y and angle). Set the bounds of the state space to [0, 25]. Create a state validator sv and bind it to the inflated map to check if the waypoints are valid. ValidationDistance is set to 0.1 meters, which represents the minimum collision check accuracy between states.

Then, Create a plannerRRTStar instance. Set the maximum number of iterations to 5000, allowing for continued expansion to optimize the path. The starting point (0.2, 3.0) is from the question requirement, which is the initial position of the vehicle and the direction angle is 0. The goal position is (13.2,16.0).

Next step, use the plan () function to search from the starting point to the end point. The generated path information is stored in pthObj and solnInfo. use tic and toc to get the time consumed for path planning (in seconds).

After that, combine solnInfo.TreeData to draw the RRT* search tree (tree structure diagram), and then overlay the actual path. Use a green circle to indicate the starting point and a blue cross to indicate the end point. The map, path, and search tree are displayed together in the diagram. Count the total number of points and total length of the path and display them and time consumed in the graph title.

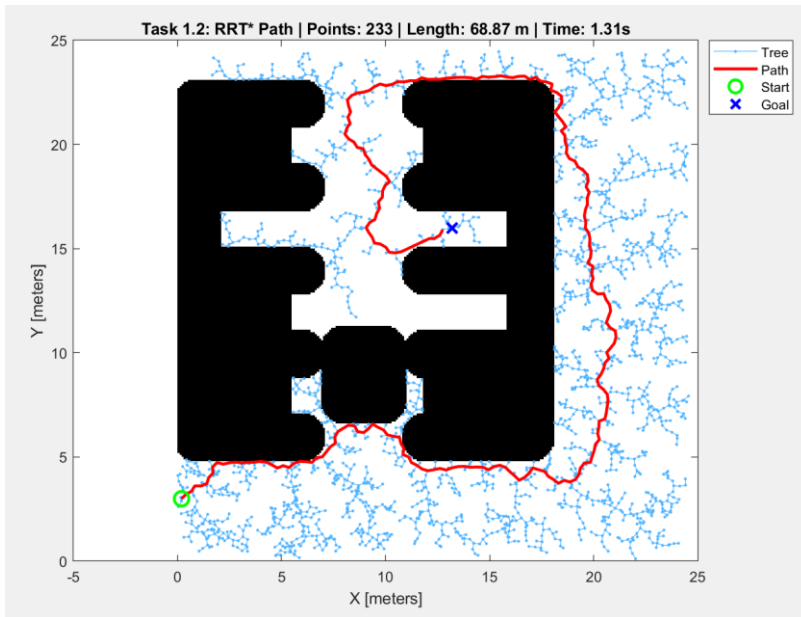


Figure 2: RRT* Path

1.3. Planning using Hybrid A*

Firstly, load the map generated in Task 1.1 and inflate it using `inflate(map, 1.1)` to take into account vehicle width and safety distance.

Secondly, use `stateSpaceSE2` to define the vehicle state (including position and orientation), and then use `validatorOccupancyMap` to bind it to the inflated map and set the validation accuracy `ValidationDistance = 0.1`

Then, initialize `plannerHybridAStar` and set parameters, and use `rng(1)` to ensure reproducibility.

Next, set the start and end points, execute the plan and time it.

Finally, use `show(planner)` to display the search process and the final path, and show the number of path points, path length, and time in the title.

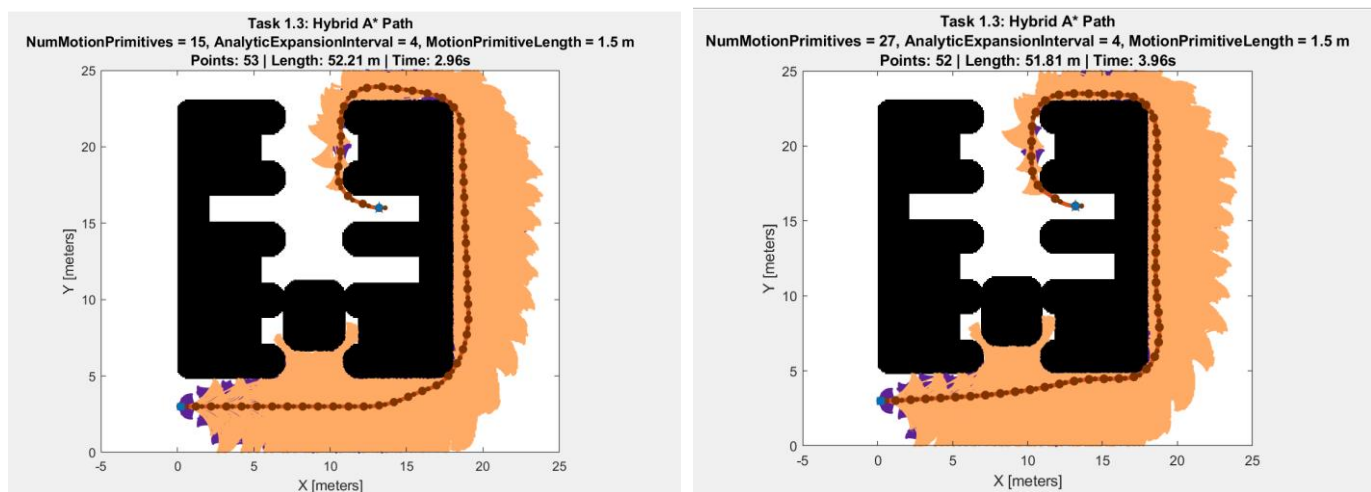


Figure 3

In conclusion, number of path points indicates how many key position points are included in the path; total path length indicates the total length of the path segments (meters), which can be used as a reference indicator for path quality; timed length indicates the time it takes to find this path. The RRT* paths are usually in the form of broken lines, which may not meet the vehicle's motion constraints, but can quickly find feasible solutions and gradually optimize.

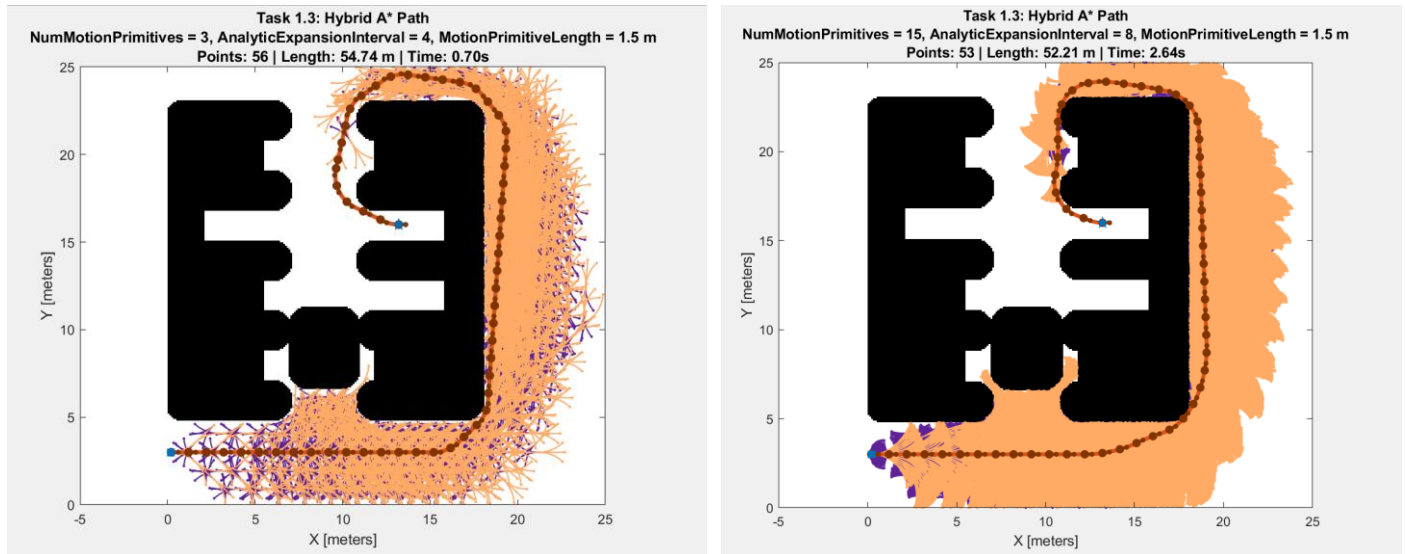


Figure 4

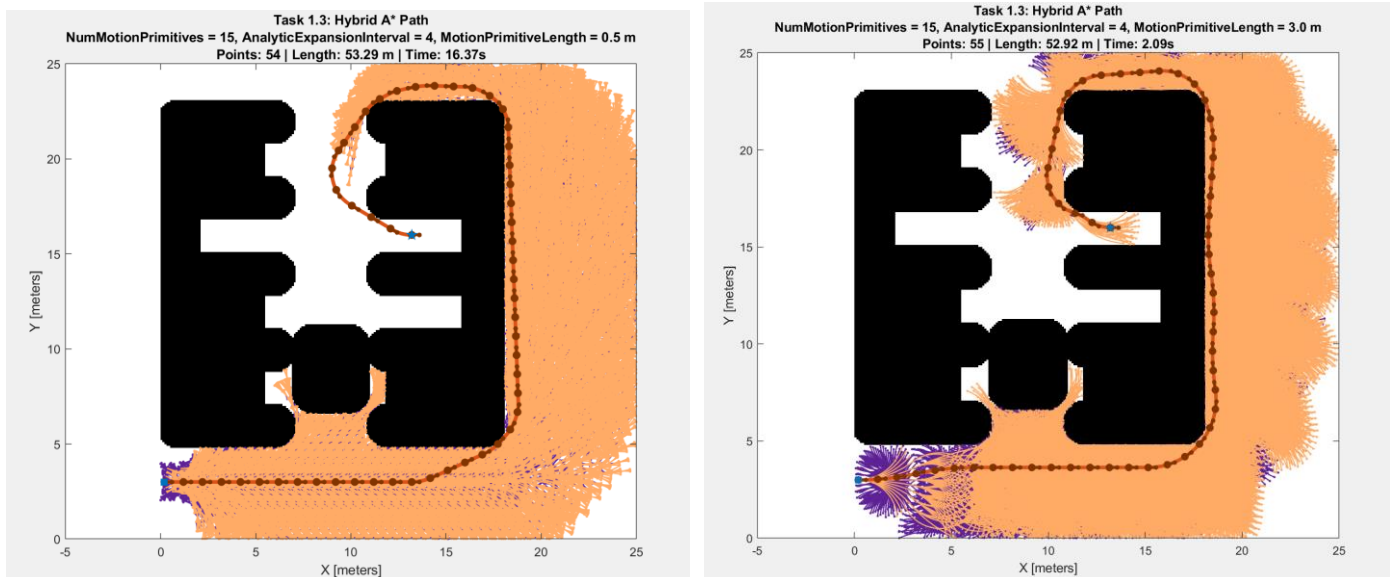


Figure 5

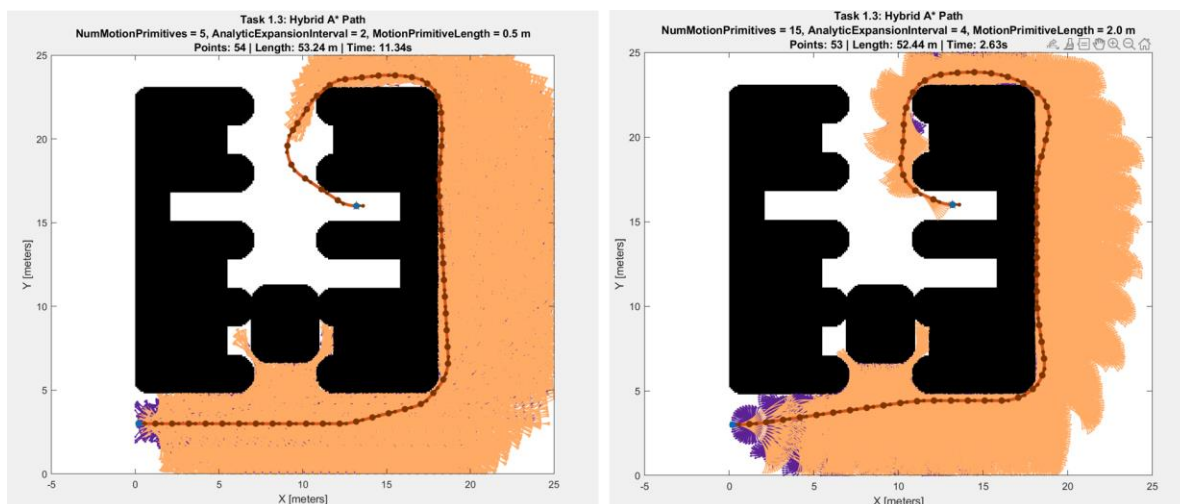


Figure 6

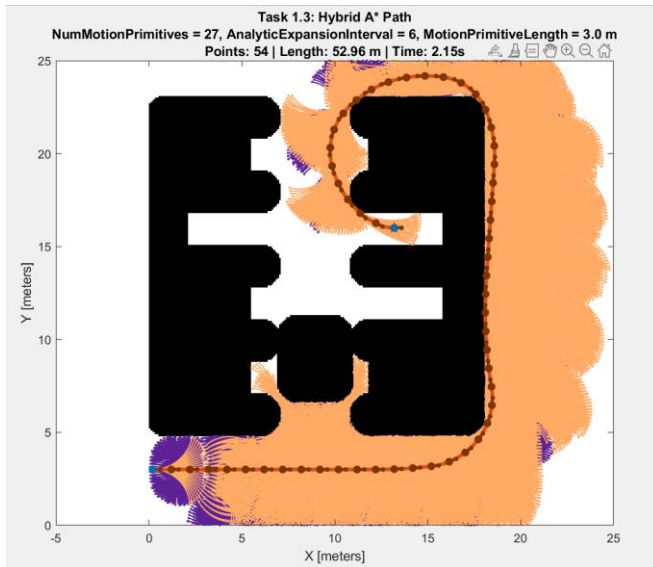


Figure 7

Conclusions drawn from the figures and tables:

Num Motion Primitives: Increasing it can help improve path flexibility, but will increase the amount of searching. As can be seen from the table, (3,4,1.5) has a short time but poor path quality, while (27,4,1.5) has a good path quality but takes a long time.

NPM Must be an odd number because intermediate values correspond to straight-line motion, with symmetrical steering angles on either side (e.g. -60° , -30° , 0° , $+30^\circ$, $+60^\circ$).

Analytic Expansion Interval: The smaller the interval, the more frequent the attempts to connect to the target point, which can increase the speed.

Motion Primitive Length:

If it is too small, the path will be too fragmented, the tree structure will be too dense, and the time cost will be huge, shown as (15,4,0.5); The middle value has the most stable effect.

(NMP,AEI,MPL)	Length of path(m)	Points	Time (s)	Established tree complexity	Path quality
(15, 4, 1.5)	52.21	53	2.96	Medium	Good
(27, 4, 1.5)	51.81	52	3.96	Medium	Good
(3, 4, 1.5)	54.74	56	0.70	Low	Bad
(15, 8, 1.5)	52.21	53	2.64	Medium	Good
(15, 4, 0.5)	53.29	54	16.37	Very High	Moderate
(15, 4, 3)	52.93	55	2.09	Medium	Moderate
(5, 2, 0.5)	53.24	54	11.34	High	Moderate
(15, 4, 2)	52.44	53	2.65	Medium	Good
(27, 6, 3)	52.96	54	2.15	Medium	Good

Table 1 Comparison of trajectory results under different (NMP, AEI, MPL) parameter settings

During adjusting the three key parameters of the Hybrid A* algorithm and compare the path length, planning time, path smoothness, and search tree complexity. The results show that NumMotionPrimitives and MotionPrimitiveLength are the core factors affecting the path, and the two need to be compromised between feasibility and efficiency. The recommended parameter combination is (15, 4, 2.0)

1.4. Discussion

1. Path Quality and Kinematic Feasibility

RRT* often produces paths with numerous sharp turns and abrupt changes in direction, resulting in jagged and non-smooth trajectories. These are typically incompatible with the non-holonomic kinematic constraints of a vehicle, such as steering angle limitations.

In contrast, Hybrid A* incorporates non-holonomic constraints, directional constraints, and the vehicle's minimum turning radius during the planning process. As a result, it generates much smoother and more continuous curves that better approximate feasible trajectories for real vehicles.

2. Computation Time

RRT* has a relatively short planning time because it uses a random sampling strategy that allows it to rapidly expand the search tree toward the goal. In this case, RRT* took approximately 1.3 seconds.

Hybrid A*, on the other hand, requires simulating motion primitives and verifying feasibility at each expansion step, which makes its planning process more time-consuming. Depending on the parameters used, the computation time for Hybrid A* ranged from 0.7 seconds to as much as 17 seconds.

2. Part 2

2.1 *Reference trajectory generation*

The objective of this task is to generate a smooth reference trajectory that satisfies vehicle kinematic constraints—for example, by inserting arc segments where necessary to avoid sharp turns. The trajectory must pass through all the specified must-pass waypoints as well as the additional auxiliary points I introduced to smooth the path. At the same time, it should maintain a safe distance from environmental obstacles. A three-phase velocity profile (acceleration – cruising – deceleration) is also applied to ensure physical feasibility and to facilitate its use in Task 2.2.

2.1.1

First, the car park map provided in the assignment is loaded, and the obstacle regions are inflated by 1.1 meters to account for the vehicle's own size. Based on this, a set of trajectory points is constructed, including: the must-pass waypoints required by the assignment, and manually inserted auxiliary points to enable smoother turns.

2.1.2

To avoid sharp turns greater than 90° in the trajectory—which are infeasible for real vehicle control—the change in heading is checked between every three consecutive waypoints p_1 , p_2 , p_3 . If the turning angle exceeds 90°, a custom function `arc_through_three` (p_1 , p_2 , p_3 , N) is called to insert an arc segment; each arc includes 10 interpolated points. After insertion, the trajectory is continuous and differentiable. This step significantly improves the smoothness and controllability of the trajectory.

2.1.3

All points that fall into obstacle regions are removed using the `checkOccupancy` function, ensuring that the entire trajectory is generated within collision-free areas.

2.1.4

The collision-free trajectory points are converted into a path parameterized by cumulative arc length. The Euclidean distance between each pair of consecutive points is computed to construct the cumulative arc length vector. Then, cubic spline interpolation is applied to $x(s)$ and $y(s)$, generating continuous spatial path functions $cs_x(s)$ and $cs_y(s)$. These functions are differentiable and can be used for subsequent velocity and heading calculations.

2.1.5

The arc-length-based trajectory $s(t)$ is mapped back to the spatial domain $x(t)$, $y(t)$ by evaluating the cubic splines $cs_x(s)$ and $cs_y(s)$ using `ppval` (), resulting in the final position trajectory. Then, the first derivatives of the splines are computed using `fnder` () to obtain the direction vectors of the trajectory. The heading angle θ_{ref} at each time step is calculated using `atan2(dy, dx)`. The first and second derivatives of the heading angle—representing angular velocity

and angular acceleration—are then computed using finite differences combined with filtering, and are denoted as ω_{ref} and α_{ref} , respectively.

2.1.6

If the turning angle between adjacent segments of the path exceeds 90° , an arc segment is inserted at that location. A custom function `arc_through_three` (p_1, p_2, p_3, N) is used to construct a circular arc that passes through the three points, with N interpolated points serving as replacement waypoints. The implementation involves the following steps: Construct an equation system to derive the centre c of the circle using the three-point method; Solve the linear system to obtain the circle centre in global coordinates; Calculate the distance from the centre to p_1 as the radius r ; Use `atan2` to compute the angles from the centre to p_1 and p_3 , then determine the total deflection angle Δ_{theta} ; Normalize the direction of rotation to ensure consistency; Use `linspace(theta1, theta2, N)` to create evenly spaced angle values; Generate arc interpolation points using polar-to-Cartesian conversion; Insert the generated `arc_pts` into the main path, replacing the original sharp-corner segment. This method ensures that the path maintains smooth curvature transitions at turns.

2.1.7

A three-phase velocity control model (acceleration – cruising – deceleration) is applied to each segment of the path using the `trapezoidal_profile()` function. This function considers the following constraints: a maximum acceleration of 2 m/s^2 , a maximum deceleration of 3 m/s^2 , and a maximum cruising speed of 3 m/s . If the remaining cruising distance S_c is negative, it means there isn't enough space for a constant-speed phase—so the vehicle must decelerate immediately after accelerating. In such cases, a quadratic equation is used to solve for V_{peak} , the highest speed that can be reached within the limited distance. Each path segment is assigned its own set of $s(t)$, $v(t)$, $a(t)$ values. Finally, all segments are concatenated to form a global time sequence.

Although auxiliary arc points are inserted for path smoothing, the speed profile is only parameterized along the main segments connecting the must-pass waypoints.

2.1.8 Plots, observations and conclusions

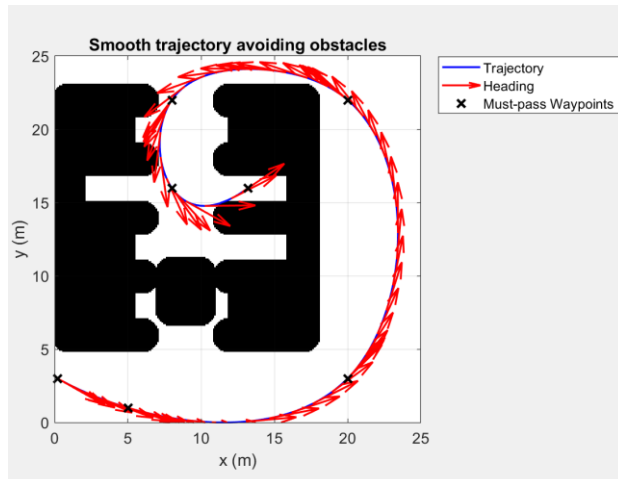


Figure 9

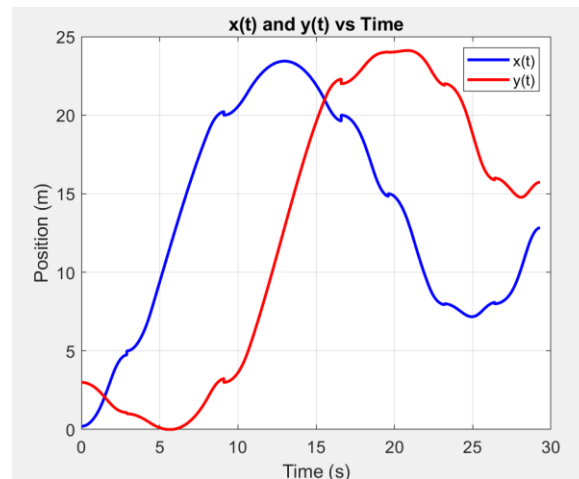


Figure 8

The reference trajectory was successfully generated by combining spatial smoothing and segment-wise velocity planning.

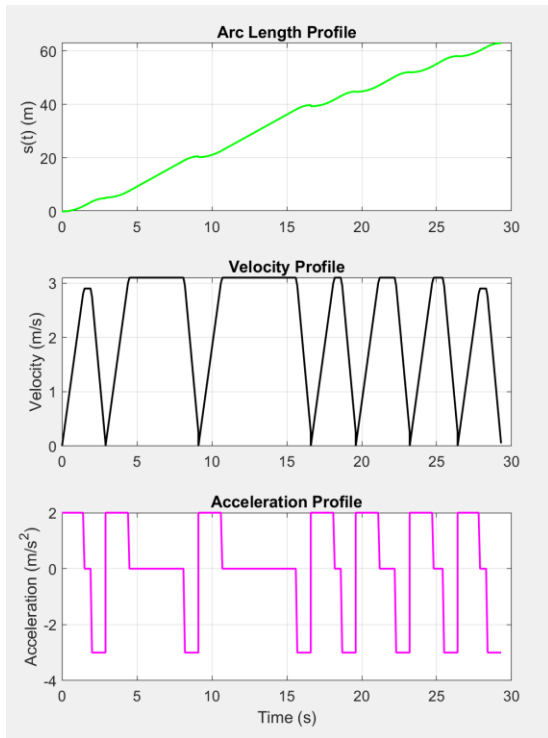


Figure 10

In figure 9 shows that the vehicle path smoothly navigates through all must-pass waypoints while maintaining safe clearance from obstacles. The heading profile is clearly visualized, showing consistent orientation alignment along the path. The figure 8 demonstrates smooth and continuous spatial transitions, with no abrupt turns or reversals. In figure 10, the velocity curve exhibits demonstrate a trapezoidal speed structure applied between each pair of must-pass waypoints, indicating a consistent acceleration–cruise–deceleration strategy. Acceleration limits (+2 to -3 m/s²) are respected, and the Total path length $s(t)$ is monotonically increasing as expected.

Overall, the generated reference trajectory satisfies both kinematic feasibility and environmental safety, and is suitable for task2.2

In Task 2.1, instead of simply connecting the must-pass waypoints with straight lines or sharp turns, I proactively applied arc interpolation and smoothing strategies to ensure that the generated path satisfies the vehicle's kinematic constraints, making it both controllable and executable.

Furthermore, I implemented local trapezoidal velocity profiles between each pair of must-pass points to produce physically feasible velocity and acceleration trajectories. The resulting reference is suitable for direct use in Task 2.2.

I believe that the RHP module should focus on optimization, fine-tuning, and enhancing robustness, rather than merely compensating for an unrealistic or infeasible reference path.

2.2 Receding-horizon planning (RHP)

2.2.1 sys_h_2.m

In Task 2.2, I reused the system dynamics model from Week 05 and named the function sys_h_2.m to distinguish it. The model itself was not modified, as it already satisfies the vehicle kinematic modelling requirements specified in the task.

2.2.2 cost_2.m

The cost.m function from Week 05 only evaluates the terminal state and does not account for the integral of running costs along the trajectory. However, according to the requirements of Task 2.2 (and as outlined in the Week 06 lecture notes), the cost function must include a running cost term in integral form. Therefore, I added an integral accumulation structure using the time step dt . A for loop iterates through each time step, and at every step, the current control cost and position tracking error are multiplied by dt and accumulated—effectively approximating the continuous-time cost using discrete integration.

```
L = 0;
for k = 1:N
    % Control effort penalty
    u_cost = 0.5 * (u(1,k)^2 + u(2,k)^2);

    % Position tracking error
    tracking_cost = wp * ((x(1,k) - r1(k))^2 + (x(2,k) - r2(k))^2);

    % Weighted Points
    L = L + (u_cost + tracking_cost) * dt;
end
```

Now we define $L(x, u)$ as

$$L(x, u) = \frac{1}{2} u_d(t)^2 + \frac{1}{2} u_a(t)^2 + w_p(r_1(t) - x_1(t))^2 + w_p(r_2(t) - x_2(t))^2,$$

Figure 11

Figure 12

u_cost represents the penalty on control inputs (acceleration and steering angle), encouraging smoother and more stable control actions. $tracking_cost$ penalizes the deviation from the reference trajectory at each time step (limited to the x and y coordinates), and is weighted by the matrix w_p . According to the assignment requirements, the terminal cost is defined as follows:

and the terminal cost as

$$H(x(t_0 + T)) = w_h(r_3(t_0 + T) - x_3(t_0 + T))^2$$

Figure 13

It is clearly stated that only the heading deviation at the terminal point needs to be penalized. Therefore, I removed the unnecessary state error terms from the original function and used the `wrapToPi()` function to handle angular wrap-around issues.

```
% Terminal cost: heading error
heading_err = wrapToPi(r3(end) - x(3,end)); %The difference between the expected angle and
                                           % the actual angle at the end
H = wh * heading_err^2; % Weighted square penalty
```

Figure 14

In the Week 05 code, the weights and terminal state are hard-coded, making them difficult to modify or extend. Since Task 2.2 requires consideration of the entire reference trajectory (x_ref) and support for adjustable weights, I encapsulated the relevant variables into a `prob_info` structure for better flexibility and modularity.

```
wp = prob_info.wp; % Position tracking error weight
wh = prob_info.wh; % Heading angle terminal error weight
```

Figure 15

2.2.3 nconst.m & Task2_2_ref.m

In Task 2.2, I defined the obstacle centres using the entire centres table. By looping through each obstacle with `for i = 1:num_circles`, I constructed an inequality constraint for each one. This enabled simultaneous avoidance of multiple obstacles, in accordance with the four circular obstacles specified in the Task 2 description page. In Task 2.2, the reference trajectory generated in Task 2.1 is reused without any modifications to the path.

2.2.4 batch_2.m

In the Week 5 code, the optimization target is a fixed goal position. In contrast, Task 2.2 adopts the complete reference trajectory generated in Task 2.1 and dynamically extracts a local reference segment in each RHP iteration for tracking optimization. This modification aligns with the task's requirement for *trajectory tracking* rather than simply reaching a fixed endpoint.

In addition, I constructed `prob_info_fixed` using the centres and radii of multiple obstacles to more realistically simulate a complex environment; I introduced the reference trajectory `x_ref_full` from `Task2_2_ref`;

In each iteration, the current local time window is determined using `idx_start` and `idx_end`, allowing the relevant segment to be extracted from the full reference trajectory for optimization. I introduced a while loop structure, which controls termination based on the distance between the current position and the target. In each loop, only the first half of the optimized trajectory is executed, and the resulting state is updated as the initial condition for the next iteration. In contrast, the Week 5 code does not include any form of dynamic updating.

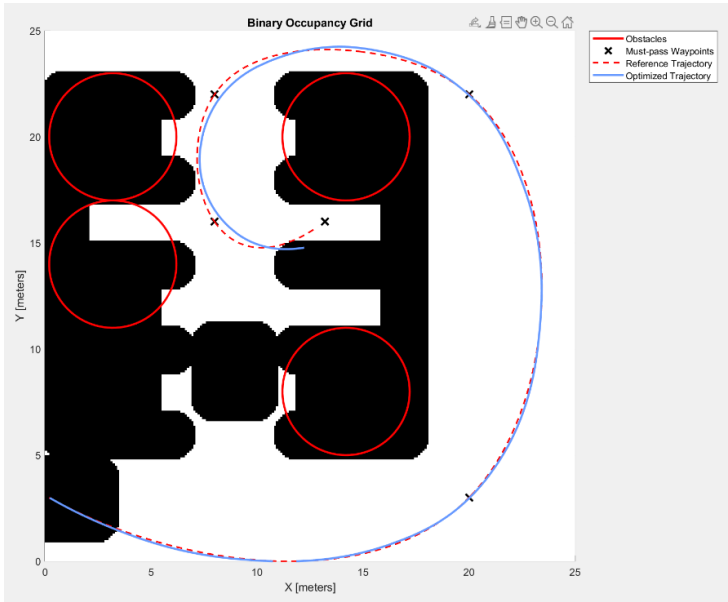
The initial guess was changed from a simple random initialization to using `repmat`, and Euler integration was employed to generate a corresponding state trajectory based on the initial control guess. This approach improves the stability and convergence of the optimizer.

I added ConstraintTolerance and OptimalityTolerance settings in the optimizer configuration to allow more stable tuning of fmincon behaviour, thereby improving the success rate of path optimization.

fprintf and disp statements were added to display the state and optimizer feedback in each iteration, making the debugging process more convenient.

A check on exitflag was added so that if the optimization fails, the loop will terminate early. This improves the system's robustness and controllability.

2.2.5



Shown as figure 16:

The optimized trajectory smoothly avoids obstacles and satisfies the nonlinear constraint with radius; it closely overlaps with the reference trajectory, indicating that tracking errors have been significantly reduced. All must-pass waypoints are successfully covered, and the reference trajectory is largely followed to completion, successfully triggering the termination condition.

Shown as figure 17:

During the RHP execution, the optimizer successfully converged in every iteration, with all tolerances meeting the task's required precision. In the 13th and final iteration, the reference trajectory had been almost entirely utilized, successfully triggering the early stopping condition.

Figure 16

```
<stopping criteria details>

Optimization completed: The relative first-order optimality measure, 8.491192e-03,
is less than options.OptimalityTolerance = 1.000000e-02, and the relative maximum constraint
violation, 8.149082e-06, is less than options.ConstraintTolerance = 5.000000e-02.
Iteration 13, distance to goal: 1.5670
Reference trajectory exhausted. Exiting loop.
Total RHP planning time: 15.05 seconds
```

Figure 17

From figure 18, Heading Plot (top right): The vehicle's heading angle changes smoothly, indicating that the terminal heading error governed by H_{terminal} is effectively controlled.

Forward speed (bottom left): The forward velocity remains within a reasonable range throughout the trajectory, never exceeding 3 m/s, which complies with the physical constraints. Although there are a few noticeable fluctuations in the middle section, they are likely due to cornering, indicating a degree of feasibility. The velocity gradually decreases toward the end, showing that the system can adjust its speed in response to the terminal point, satisfying the convergence requirement.

Control Inputs Plot (bottom right): The steering input remains within the range $[-0.5, 0.5]$, satisfying the input constraint requirements of the task. However, there are noticeable jumps and oscillations during multiple time intervals—especially in the acceleration signal, which exhibits more pronounced fluctuations. This reflects a certain level of control intensity and frequent adjustments within the system.

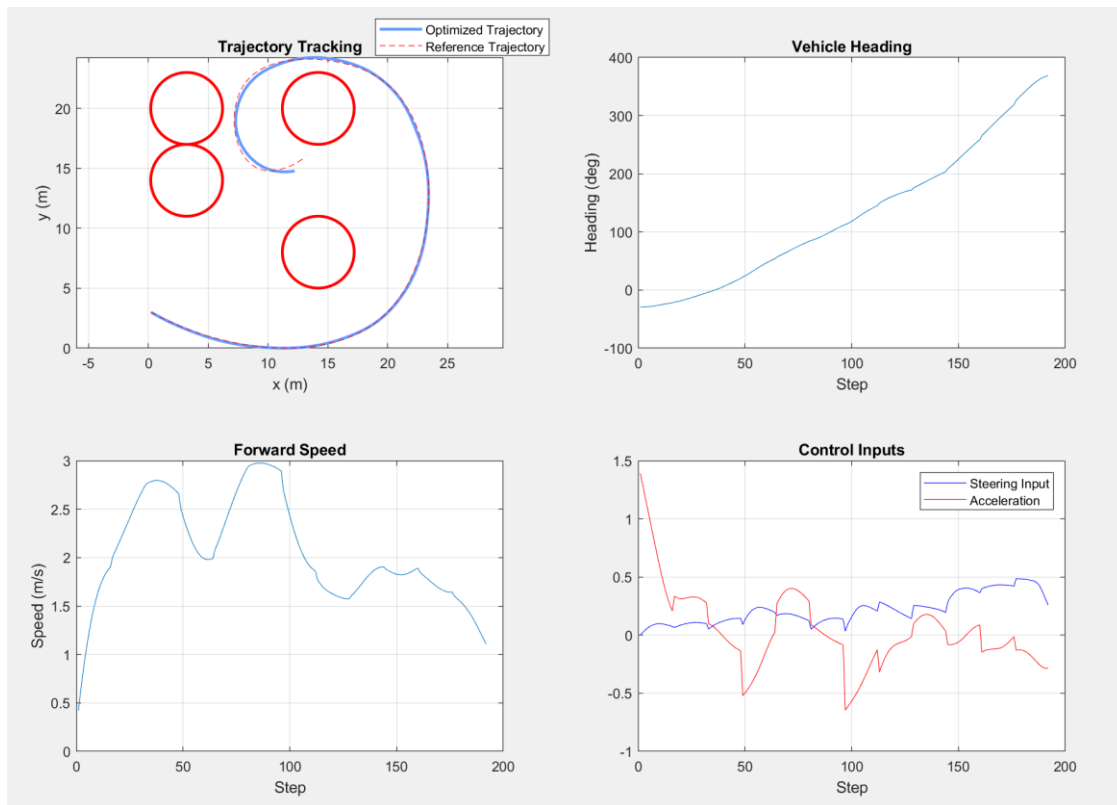


Figure 18

2.3 Discussion about RHP trajectory

2.3.1

The RHP trajectory performance, convergence behaviour, stopping conditions, challenges encountered, etc. have been **comprehensively discussed in Section 2.2.5**; this section focuses on the algorithmic improvements.

Fine-tuned the optimality tolerance of `fmincon` from $1e-2$ to $5e-2$, this adjustment shortens the time required for each round of optimization: Total RHP planning time: 10.25 seconds

2.3.2

Quality of the planned trajectories: The RHP trajectory is geometrically smoother and more continuous; in contrast, the Hybrid A* trajectory does not closely follow the edges of the feasible region and requires post-processing before it can be used for control.

Computation time: Hybrid A* has a shorter computation time (under 4 seconds), making it suitable for quickly generating an initial solution. RHP involves longer optimization times per iteration, and the total time depends on the number of iterations—typically taking several seconds to over a dozen seconds in total.

Merits and demerits of each approach:

Method	Merits	Demerits
Hybrid A*	Fast, suitable for initial path generation	Not smooth enough
RHP	Smooth, kinematically feasible, and usable for real vehicle control	The calculation is time-consuming and depends on the initial guess and reference trajectory.

Appendix 1: Codes***Task1_1.m:***

```

% Task 1.1
% Initialize map parameters

mapWidth = 25;    % Map Width
mapHeight = 25;   % Map Height
resolution = 10;  % Each grid represents 0.1 meters

% Create empty binaryOccupancyMap
map = binaryOccupancyMap(mapWidth, mapHeight, resolution);

% Draw the stop lines
% Parking space border line on the left
% Bottom border, outer border, top border
for y = [6, 10, 14, 18]
    setOccupancy(map, [1:0.1:6; y*ones(1, length(1:0.1:6))]', 1);
end
setOccupancy(map, [1*ones(1, length(6:0.1:22)); 6:0.1:22]', 1);
setOccupancy(map, [1:0.1:6; 22*ones(1, length(1:0.1:6))]', 1);

% Parking space border line on the right
for y = [6, 10, 14, 18]
    setOccupancy(map, [12:0.1:17; y*ones(1, length(12:0.1:17))]', 1);
end
setOccupancy(map, [17*ones(1, length(6:0.1:22)); 6:0.1:22]', 1);
setOccupancy(map, [12:0.1:17; 22*ones(1, length(12:0.1:17))]', 1);

% Drawing obstacle vehicles
markRectangle(map, 2, 7.1, 2.4, 1.8); % left 3 cars
markRectangle(map, 2, 11.1, 2.4, 1.8);
markRectangle(map, 2, 19.1, 2.4, 1.8);

markRectangle(map, 13, 7.1, 2.4, 1.8); % right 2 cars
markRectangle(map, 13, 19.1, 2.4, 1.8);

markRectangle(map, 8.1, 7.8, 1.8, 2.4); % middle car

% Visual Map

figure;
show(map)
title('Task 1.1: Binary Occupancy Map')

```

```

xlabel('X (m)')
ylabel('Y (m)')
xlim([-5, 25])
ylim([0, 25])

save('task1_1_map.mat', 'map')

% "Brush a rectangle" on the map to represent an obstacle vehicle
function markRectangle(map, x, y, w, h)

    xs = x : 0.05 : (x + w);      % From left to right
    ys = y : 0.05 : (y + h);      % From bottom to top
    [X, Y] = meshgrid(xs, ys);
    setOccupancy(map, [X(:), Y(:)], 1); % Set the entire rectangular area to be occupied
end

```

Task1_2.m

```

% Task1.2
% isValidStart = isValidState(sv, start)
% Visual verification of the starting point
% figure;
% show(inflatedMap)
% hold on
% plot(start(1), start(2), 'ro', 'MarkerSize', 10, 'LineWidth', 2)
% title('Check if Start State is Valid')

% load the occupancy map generated in Task 1.1
load('task1_1_map.mat')

% Expand the obstacle boundary, considering the vehicle size
% here expand 1.1 meters
inflatedMap = copy(map);
inflate(inflatedMap, 1.1); % 1.1m

% set up the state space (2D plane)
ss = stateSpaceSE2;
ss.StateBounds = [0 25; 0 25; -pi pi]; % Define the range of x, y, theta

% Create stateValidator
% Used to detect whether the state in the path is in free space
sv = validatorOccupancyMap(ss);
sv.Map = inflatedMap;
sv.ValidationDistance = 0.1; % Minimum detection interval between states

% RRT* planner
planner = plannerRRTStar(ss, sv);

```

```

planner.ContinueAfterGoalReached = true;    % Continue to optimize the path after finding the
target
planner.MaxIterations = 5000;               % Maximum number of iterations
planner.MaxConnectionDistance = 0.3;       % The maximum connection distance between nodes
                                           % (affects the branch density of the tree)

% Set starting and goal positions
start = [0.2, 3.0, 0];
goal  = [13.2, 16.0, 0];

% Planning
rng(0); % Fixed random number seed to ensure reproducible results
tic;    % Start timing
[pthObj, solnInfo] = plan(planner, start, goal);
planningTime = toc;    % Get the time taken for path planning (in seconds)

% Show the figure
figure;
show(inflatedMap);
hold on;

% Show the established tree
plot(solnInfo.TreeData(:,1), solnInfo.TreeData(:,2), '-.', ...
     'Color', [0.3, 0.7, 1.0], 'LineWidth', 0.5);

% Show paths
plot(pthObj.States(:,1), pthObj.States(:,2), 'r-', 'LineWidth', 2)

% Mark start point and goal point
plot(start(1), start(2), 'go', 'MarkerSize', 10, 'LineWidth', 2)
plot(goal(1), goal(2), 'bx', 'MarkerSize', 10, 'LineWidth', 2)

% Number of points and length of path
numPoints = size(pthObj.States, 1);
pathLength = sum(vecnorm(diff(pthObj.States(:,1:2)), 2, 2));

% Title
title(sprintf('Task 1.2: RRT* Path | Points: %d | Length: %.2f m | Time: %.2fs', ...
             numPoints, pathLength, planningTime));
xlim([-5 25])
ylim([0 25])
legend('Tree', 'Path', 'Start', 'Goal')

```

Task1_3.m

% Task1.3

```
% Load the occupancy map from Task 1.1
load('task1_1_map.mat')

% Create a copy of the map and inflate it
inflatedMap = copy(map);
inflate(inflatedMap, 1.1);

% Define the state space (x, y, orientation)
ss = stateSpaceSE2;
ss.StateBounds = [0 25; 0 25; -pi pi]; % Set state boundaries

% Create a state validator
sv = validatorOccupancyMap(ss); % Create a map validator corresponding to the state space
sv.Map = inflatedMap;          % Use the inflated map for collision detection
sv.ValidationDistance = 0.1;    % To improve accuracy

% Set up the Hybrid A* planner
planner = plannerHybridAStar(sv);

% Try adjusting the following three parameters
planner.NumMotionPrimitives = 15;
planner.AnalyticExpansionInterval = 4;
planner.MotionPrimitiveLength = 2;

% Set the start and end points
start = [0.2, 3.0, 0];
goal = [13.2, 16.0, 0];

% Set a random number
% pthObj: path object
% solnInfo: including path search tree information
% tic and toc for timing
rng(1); tic;
[pthObj, solnInfo] = plan(planner, start, goal);
planningTime = toc;

% Extract the state point sequence of the path
% vecnorm() represents Euclidean distance
% sum() gets the total length of the path
numPoints = size(pthObj.States, 1);
pathLength = sum(vecnorm(diff(pthObj.States(:,1:2))), 2, 2));

% Plot with tree and path
% All expanded nodes (orange arrows)
```



```
% Final path (brown-red line)
% Add title
figure;
show(planner);
title(sprintf(['Task 1.3: Hybrid A* Path\n' ...
    'NumMotionPrimitives = %.0f, AnalyticExpansionInterval = %.0f, MotionPrimitiveLength = %.1f\n' ...
    'Points: %d | Length: %.2f m | Time: %.2fs'], ...
    planner.NumMotionPrimitives, ...
    planner.AnalyticExpansionInterval, ...
    planner.MotionPrimitiveLength, ...
    numPoints, pathLength, planningTime));

xlim([-5, 25])
ylim([0, 25])
```

Task2_1_ref.m

% Task2.1 - Reference trajectory generation

```
function [x_ref, y_ref, theta_ref, s_ref, v_ref, a_ref, omega_ref, alpha_ref, t_ref] =
Task2_1_ref(dt)
% This function generates a smooth, kinematically feasible reference trajectory
% based on a set of must-pass waypoints. It includes:
% - x_ref, y_ref: interpolated global positions along the trajectory
% - theta_ref: heading direction
% - v_ref, a_ref: velocity and acceleration profile
% - omega_ref, alpha_ref: angular velocity and acceleration
% The trajectory avoids sharp turns by inserting arc segments where needed
% It also applies a three-stage velocity profile (acceleration - cruise - deceleration) to ensure
physical feasibility
% This trajectory will later be used in Task2.2 as well

dt = 0.1;

% === Load the map and inflate it to account for vehicle size ===
load('task1_1_map.mat');
inflatedMap = copy(map);
inflate(inflatedMap, 1.1);

% === Set the waypoints, including auxiliary points for steering control ===
wp_base = [0.2  5   20  20  15   8   8  13.2;
```

```

        3    1    3    22    24    22    16    16];
% Each column represents an (x, y) coordinate

% === Smoothing with arcs ===
% Use arc interpolation to smooth the path and avoid situations such as
% 90-degree blind spots that do not conform to vehicle kinematics
arc_resolution = 10;          % Insert 10 interpolation points into each arc
angle_thresh = pi/2;          % If the angle is greater than 90 degrees, perform arc smoothing
wp_aug = wp_base(:,1);        % Initialize the smoothed path, starting from the first point

for i = 2:size(wp_base,2)-1    % Traverse each group of 3 points in the path to determine whether
    an arc segment needs to be inserted
        p0 = wp_base(:,i-1);    % Previous point
        p1 = wp_base(:,i);      % Current point
        p2 = wp_base(:,i+1);    % Next point
        v1 = p1 - p0;           % Vector: from p0 to p1
        v2 = p2 - p1;           % Vector: from p1 to p2
        theta1 = atan2(v1(2), v1(1)); % Angle of the first vector
        theta2 = atan2(v2(2), v2(1)); % Angle of the second vector
        delta_theta = wrapToPi(theta2 - theta1); % Angle, converted to [-pi, pi]
        if abs(delta_theta) > angle_thresh % If it is a sharp turn, insert an arc
            segment between these three points
                arc_pts = arc_through_three(p0, p1, p2, arc_resolution);
                wp_aug = [wp_aug, arc_pts(:,2:end-1), p2];
                % The inserted points exclude the first and last ones to avoid duplication
            else
                % Otherwise directly connect to the current point
                wp_aug = [wp_aug, p1];
            end
        end
    end

% Ensure that the end target point is added
if ~isequal(wp_aug(:,end), wp_base(:,end))
    wp_aug = [wp_aug, wp_base(:,end)];
end

% === Occupancy check ===
valid = ~checkOccupancy(inflatedMap, wp_aug');
wp = wp_aug(:,valid);

% === s → x(s), y(s) ===
dist = sqrt(sum(diff(wp,1,2).^2)); % Calculate the length of each path
s_wp = [0, cumsum(dist)];          % Cumulative path length,
cs_x = spline(s_wp, wp(1,:));      % Interpolate x(s) using cubic spline
cs_y = spline(s_wp, wp(2,:));      % Interpolate y(s) using cubic spline

```

```

% === Use local three-stage speed control ===

% Set the maximum speed, acceleration, and deceleration
vmax = 3; amax = 2; dmax = 3;
s_all = []; v_all = []; a_all = []; t_all = []; % Initialize all track quantities
s_offset = 0; t_offset = 0; % Cumulative offset, used to splice each
segment
for i = 1:length(dist)
    s_i = dist(i); % The length of the current segment
    % Use the trapezoidal profile function to calculate the time, s(t), v(t), and a(t) of this
    trajectory
    [t_seg, s_seg, v_seg, a_seg] = trapezoidal_profile(s_i, 0, 0, vmax, amax, -dmax, dt);
    s_all = [s_all, s_seg + s_offset]; % Global s(t), velocity, acceleration, deceleration,
    time,
    v_all = [v_all, v_seg];
    a_all = [a_all, a_seg];
    t_all = [t_all, t_seg + t_offset];
    s_offset = s_offset + s_i; % Update path length offset
    t_offset = t_offset + t_seg(end); % Update time offset
end

% === Interpolated trajectory ===
% Insert the cumulative path length s_all into the previously constructed x(s) and y(s) curves
% to obtain the coordinate position corresponding to each time step on the trajectory
x_ref = ppval(cs_x, s_all);
y_ref = ppval(cs_y, s_all);

% === Filter obstacles ===
% Prevent the trajectory from crossing obstacles and remove every unreasonable point
valid2 = ~checkOccupancy(inflatedMap, [x_ref(:), y_ref(:)]);
x_ref = x_ref(valid2);
y_ref = y_ref(valid2);
s_ref = s_all(valid2);
t_ref = t_all(valid2);
v_ref = v_all(valid2);
a_ref = a_all(valid2);

% === Headings ===
% Calculate heading angle (theta), angular velocity (omega), angular acceleration (alpha)
% The heading angle theta represents the direction the vehicle is facing at this moment
% Angular velocity omega indicates the speed of turning; angular acceleration alpha indicates the
trend of turning

dx = ppval(fnder(cs_x), s_ref);
dy = ppval(fnder(cs_y), s_ref);
theta_ref = atan2(dy, dx);
omega_ref = [0 diff(theta_ref)/dt]; % Calculate angular velocity (difference approximation)

```

```

omega_ref = sgolayfilt(omega_ref, 3, 11); % Use Savitzky-Golay filter to smooth the angular
velocity
alpha_ref = [0 diff(omega_ref)/dt];

end

function arc_pts = arc_through_three(p1, p2, p3, N)
% Input: p1, p2, p3 are the coordinates of three points, and the arc is required to pass through
these three points
% N represents the number of interpolation points generated
% Output: arc_pts is a 2xN array, each column is a point on the arc after interpolation

% The method of geometric three-point circle determination to solve the center c
A = 2 * [p2'-p1'; p3'-p1'];
b = [norm(p2-p1)^2; norm(p3-p1)^2];
c = (A\b) + p1'; % Find the center position c
r = norm(c - p1); % Radius r, which is the distance from the center of the circle to
any point
theta1 = atan2(p1(2)-c(2), p1(1)-c(1)); % p1 polar angle relative to the center of the circle
theta3 = atan2(p3(2)-c(2), p3(1)-c(1)); % p3 Polar angle relative to the center of the circle
dtheta = wrapToPi(theta3 - theta1); % Angle Difference, use wrapToPi to wrap it to the
range [-pi, pi]

if dtheta < 0, dtheta = dtheta + 2*pi; end % If it is a negative angle, it becomes positive
(clockwise)
theta = linspace(theta1, theta1 + dtheta, N); % Generate N points starting from theta1 with
uniform angle steps
x_arc = c(1) + r * cos(theta); % Polar coordinates converted to Cartesian coordinates
y_arc = c(2) + r * sin(theta);
arc_pts = [x_arc; y_arc]; % Returns the arc point set
end

function [t_grid, s, v, a] = trapezoidal_profile(S, v0, v1, v_max, a_pos, a_neg, dt)
% Input: path length S, starting speed v0, ending speed v1
% Maximum speed v_max, acceleration a_pos, deceleration a_neg
% Time step dt
% Output: t_grid: time series; s(t): position; v(t): speed; a(t): acceleration

ta = (v_max - v0)/a_pos; % Acceleration phase time
td = (v_max - v1)/abs(a_neg); % Deceleration phase time
sa = v0*ta + 0.5*a_pos*ta^2; % Distance traveled during the acceleration phase
sd = v_max*td + 0.5*a_neg*td^2; % Distance traveled during deceleration phase
sc = S - sa - sd; % The remaining distance is used for the constant speed stage

```

```

if sc < 0
% If the distance is too short to reach the maximum speed, use the triangular speed curve that
"removes the constant speed stage"

    A = 1/a_pos + 1/abs(a_neg);          % Solve for the most reasonable intermediate speed
v_peak
    B = -v0/a_pos - v1/abs(a_neg);
    C = (v0^2)/(2*a_pos) + (v1^2)/(2*abs(a_neg)) - S;
    v_peak = (-B + sqrt(B^2 - 4*A*C))/(2*A);
    ta = (v_peak - v0)/a_pos;
    td = (v_peak - v1)/abs(a_neg);
    tc = 0; v_max = v_peak;              % No constant speed segment
else
    tc = sc / v_max;                     % Constant speed phase time
end
T = ta + tc + td;                        % Total Time
t_grid = 0:dt:T;                         % Timeline
a = zeros(size(t_grid));                 % Initialize acceleration
a(t_grid<=ta) = a_pos;                   % Acceleration segment
a(t_grid>ta & t_grid<=ta+tc) = 0;        % Constant speed segment
a(t_grid>ta+tc) = a_neg;                 % Deceleration section
v = v0 + cumtrapz(t_grid, a);            % Integrate to get the velocity v(t)
s = v0*t_grid + cumtrapz(t_grid, v);     % Integrate again to get the position s(t)
end

```

Task2_1_plot.m

% Task2_1 Plot

```

function Task2_1_plot()

% === Load the inflated map to show the obstacle area ===
load('task1_1_map.mat');
inflatedMap = copy(map);
inflate(inflatedMap, 1.1);

% === Call Task2_1_ref.m to obtain the variables of the reference trajectory ===
dt = 0.1;
[x_ref, y_ref, theta_ref, s_ref, v_ref, a_ref, omega_ref, alpha_ref, t_ref] = Task2_1_ref(dt);

% Plot the trajectory with map background
figure;
show(inflatedMap); hold on;              % Display map background
title('Smooth trajectory avoiding obstacles');
xlabel('x (m)'); ylabel('y (m)');
set(gca, 'FontSize', 12);
grid on;

```

```

% Draw the trajectory curve (blue line)
plot(x_ref, y_ref, 'b-', 'LineWidth', 1.5);

% Draw the heading arrows (red arrows, every 5 points)
arrow_interval = 5;
quiver(x_ref(1:arrow_interval:end), y_ref(1:arrow_interval:end), ...
       cos(theta_ref(1:arrow_interval:end)), sin(theta_ref(1:arrow_interval:end)), ...
       0.8, 'r', 'LineWidth', 1.5, 'MaxHeadSize', 1);

% Must-pass waypoints (black cross)
wp_must = [0.2  5   20  20   8   8   13.2;
           3   1   3   22  22  16   16];
plot(wp_must(1,:), wp_must(2,:), 'kx', 'MarkerSize', 9, 'LineWidth', 2);

legend({'Trajectory', 'Heading', 'Must-pass Waypoints'}, 'Location', 'northeastoutside');

% Plot x(t) and y(t) position versus time
figure;
plot(t_ref, x_ref, 'b', 'LineWidth', 2); hold on;
plot(t_ref, y_ref, 'r', 'LineWidth', 2);
title('x(t) and y(t) vs Time');
xlabel('Time (s)');
ylabel('Position (m)');
legend('x(t)', 'y(t)');
set(gca, 'FontSize', 12);
grid on;

% Plot arc length, velocity, and acceleration versus time
figure;
set(gcf, 'Position', [100 100 600 900]); % Set figure size nicely
subplot(3,1,1);
plot(t_ref, s_ref, 'g', 'LineWidth', 1.5);
ylabel('s(t) (m)');
title('Arc Length Profile');
set(gca, 'FontSize', 12);
grid on;

subplot(3,1,2);
plot(t_ref, v_ref, 'k', 'LineWidth', 1.5);
ylabel('Velocity (m/s)');
title('Velocity Profile');
set(gca, 'FontSize', 12);
grid on;

subplot(3,1,3);

```

```

plot(t_ref, a_ref, 'm', 'LineWidth', 1.5);
ylabel('Acceleration (m/s^2)');
xlabel('Time (s)');
title('Acceleration Profile');
set(gca, 'FontSize', 12);
grid on;

end

```

Task2.2

sys_h_2.m

```
% sys_h_2.m
```

```

function dxdt = sys_h_2(x, u, prob_info)
% SYS_H: System dynamics function for vehicle motion.
% Inputs:
%   x: current state [x; y; theta; v; cost]
%   u: current control input [delta; a]
%   prob_info: struct of problem settings
% Output:
%   dxdt: time derivative of the state

L = 2; % length from rear wheel axle to the front axle

dxdt = [x(4,1)*cos(x(3,1)); ... % x_dot, the speed of the vehicle in the x direction
        x(4,1)*sin(x(3,1)); ... % y_dot, the speed of the vehicle in the y direction
        (x(4,1)/L)*tan(u(1,1)); ... % theta_dot, the rate of change of the heading angle,
                                     % which indicates how the vehicle is turning
        u(2,1); ... % v_dot, acceleration
        0.5*u(1,1)^2 + 0.5*u(2,1)^2]; % cost accumulation, this represents "control effort":
the larger the % angle and acceleration, the higher the cost (0.5 is
the weight).
end

```

cost_2.m

```
% cost_2.m
```

```

function value = cost_2(eta, prob_info)
% =====
% This cost function ensures that u(t) is as close as possible to:

```

```

% - Let the vehicle move along the reference trajectory (minimize x/y deviation)
% - Control as smooth as possible (avoid sharp)
% - The end point is aligned with the target angle accurately
% =====
% Returns the total cost corresponding to the given trajectory eta,
% which includes two parts:
% - Running Cost: Tracking Error + Control Effort
% - Terminal Cost: Heading Angle Error

% Get information such as the number of discrete points, number of state variables,
% number of control variables, total duration
N = prob_info.N;
Nx = prob_info.Nx;
Nu = prob_info.Nu;
T = prob_info.T;
dt = T / N;
% Error weighting factor
wp = prob_info.wp;      % Position tracking error weight
wh = prob_info.wh;      % Heading angle terminal error weight

% Unpack the front part of eta into the state matrix x, size is 5*(N+1)
% Unpack the latter part into the control matrix u, size is 2*N
x = reshape(eta(1:(N+1)*Nx), Nx, N+1);
u = reshape(eta((N+1)*Nx+1:end), Nu, N);

% Reference trajectory
x_ref = prob_info.x_ref; % size is [3 * (N+1)] => [x_ref; y_ref; theta_ref]
r1 = x_ref(1,:);         % x-coordinate reference
r2 = x_ref(2,:);         % y-coordinate reference
r3 = x_ref(3,:);         % theta angle reference

% Running cost: tracking error + control effort
L = 0;
for k = 1:N
    % Control effort penalty
    u_cost = 0.5 * (u(1,k)^2 + u(2,k)^2);

    % Position tracking error
    tracking_cost = wp * ((x(1,k) - r1(k))^2 + (x(2,k) - r2(k))^2);

    % Weighted Points
    L = L + (u_cost + tracking_cost) * dt;
end

```



```
% Terminal cost: heading error
heading_err = wrapToPi(r3(end) - x(3,end)); %The difference between the expected angle and
                                         % the actual angle at the end
H = wh * heading_err^2;      % Weighted square penalty

% Total cost
value = L + H;
end
```

nconst_2.m

```
% nconst_2.m

function [ineqvalue, eqvalue] = nconst_2(eta, prob_info)
% =====
% There are two main types of constraints:
% - Equality constraints:
%   Used to enforce system dynamics
%   All time steps require that the system state advances correctly according to
%   the dynamics.
% - Inequality constraint:
%   Used for obstacle avoidance. Ensure that the vehicle will not enter the radius of the
%   circular obstacle at any time
% =====

% Extract the configuration parameters from prob_info
N = prob_info.N;      % Optimization steps
Nx = prob_info.Nx;    % Number of state variables
Nu = prob_info.Nu;    % Number of control variables
T = prob_info.T;      % Total length of time domain
x0 = prob_info.x0;    % Initial state
delta = T / N;        % Single step duration

% Extract and reshape states and controls from eta
x = reshape(eta(1:N*Nx), Nx, N);
x = [x0, x];
u = reshape(eta((N+1)*Nx+1:end), Nu, N);

% Equality constraints: System dynamics
% For each time step k, calculate the difference in system dynamics:
%  $x(k+1) = x(k) + \delta * f(x(k), u(k))$ 
eqvalue = zeros(N*Nx, 1);
for k = 1:N
    eqvalue((k-1)*Nx+1:k*Nx, 1) = x(:,k+1) - x(:,k) - delta * sys_h_2(x(:,k), u(:,k), prob_info);
end
```

```

% Inequality constraints: Obstacle avoidance
r = prob_info.radius;           % Fixed obstacle radius
circles = prob_info.centres;     % Each center [cx cy]
num_circles = size(circles, 1); % Number of obstacles
ineqvalue = zeros(N * num_circles, 1); % Initialize inequality constraint values

% For each obstacle + each time step, construct a distance constraint:
% Ensure that the Euclidean distance between the vehicle and the center of the circle is  $\geq r$ 
for i = 1:num_circles
    cx = circles(i,1);
    cy = circles(i,2);
    for k = 1:N
        idx = (i-1)*N + k;

        ineqvalue(idx) = r^2 - (x(1,k+1) - cx)^2 - (x(2,k+1) - cy)^2;
    end
end

end
end

```

batch_2.m

```

% batch_2.m
% =====
% This part performs dynamic tracking of the reference trajectory generated in Task 2.1 using
% Receding Horizon Planning
% In each iteration, a local segment of the reference trajectory is extracted, and `fmincon` is
% called to solve a finite-horizon optimization problem
% The system state is then updated and the optimized segment is appended to the overall trajectory
% This process continues until the reference trajectory is exhausted

clear; clc;

% Parameters
N = 32;           % Number of discretisation intervals
Nx = 5;           % Number of state variables
Nu = 2;           % Number of input variables
T = 5;            % Seconds
dt = T / N;       % Step size

xg = [13.2; 16];  % Goal position (x, y)
d_goal = 0.5;     % Stopping distance threshold

% Load full reference trajectory
% The initial state is taken from the first point of the reference trajectory

```

```

% and supplemented with the initial cost (which is 0)

[x_ref_full, y_ref_full, theta_ref_full, s_ref_full, v_ref_full, a_ref_full, omega_ref_full,
alpha_ref_full, t_ref_full] = Task2_2_ref(dt);

x0 = [x_ref_full(1); y_ref_full(1); theta_ref_full(1); v_ref_full(1); 0];

% Containers for full trajectory
x_all = [];
u_all = [];
t_all = [];
t0 = 0;

% Fixed problem info
prob_info_fixed = struct('T', T, 'N', N, 'Nu', Nu, 'Nx', Nx, ...
    'wp', 0.1, 'wh', 1, ...
    'centres', [3.2 14.0; 3.2 20.0; 14.2 8.0; 14.2 20.0], 'radii', 3);

% Set variables to record the trajectory of the entire running process,
% limit the steering and acceleration range of each step, and splice
lb_control = -inf(Nu,N); ub_control = inf(Nu,N);
lb_state = -inf(Nx,N); ub_state = inf(Nx,N);
lb_control(1,:) = -0.5; ub_control(1,:) = 0.5;
lb_control(2,:) = -3; ub_control(2,:) = 2;

lb = [-inf((N+1)*Nx,1); lb_control(:)];
ub = [ inf((N+1)*Nx,1); ub_control(:)];

% Controlling the Behavior of fmincon
% Constraints do not have to be exactly 0
options = optimoptions('fmincon', ...
    'Display', 'iter-detailed', ...
    'MaxIterations', 300, ...
    'MaxFunctionEvaluations', 1e5, ...
    'ConstraintTolerance', 5e-2, ...
    'OptimalityTolerance', 1e-2);
% The setting is not very small in order to make the optimization process end faster

% Give fmincon a reasonable guess in advance to speed up convergence
init_control = repmat([0.01; 0.5], 1, N);

% ===== Main RHP Loop =====
max_iter = 100;
iter = 0;

tic;

```

```

% End condition: the distance between the current car position and the target position is less
than the threshold
while (x0(1) - xg(1))^2 + (x0(2) - xg(2))^2 > d_goal^2 && iter < max_iter
    iter = iter + 1;
    % Displays the current number of iterations and the Euclidean distance from the target point.
    This is convenient for debugging
    dist2goal = norm(x0(1:2) - xg);
    fprintf("Iteration %d, distance to goal: %.4f\n", iter, dist2goal);

    % The starting index and ending index of the current optimization window
    idx_start = max(1, round(t0/dt) + 1);
    idx_end = min(length(t_ref_full), idx_start + N);

    % Determine whether the reference trajectory has been reached
    if idx_start >= length(x_ref_full) - 1
        disp('Reference trajectory exhausted. Exiting loop.');
```

break;

```

    end

    fprintf("Ref segment index: %d ~ %d / %d\n", idx_start, idx_end, length(x_ref_full));

    % Extract the reference path segment within the current time window from the complete
    reference trajectory
    % for this round of optimization
    x_rhp = x_ref_full(idx_start:idx_end);
    y_rhp = y_ref_full(idx_start:idx_end);
    theta_rhp = theta_ref_full(idx_start:idx_end);

    % And padding is performed on the insufficient length to ensure that there are N+1 steps
    if isempty(x_rhp)
        x_rhp = zeros(1, N+1);
        y_rhp = zeros(1, N+1);
        theta_rhp = zeros(1, N+1);
    elseif length(x_rhp) < N+1
        x_rhp(end+1:N+1) = x_rhp(end);
        y_rhp(end+1:N+1) = y_rhp(end);
        theta_rhp(end+1:N+1) = theta_rhp(end);
    end

    % prob_info needs to be updated every round because the starting point x0 will change
    % and the reference trajectory x_ref also needs to use local segments
    prob_info = prob_info_fixed;
    prob_info.x0 = x0;
    prob_info.x_ref = [x_rhp; y_rhp; theta_rhp];

```

```

% Initial guess trajectory
% Use Euler's method to integrate the system state, generate the initial state trajectory, and
then superimpose it with the initial
% control to form the optimization variable
init_state = zeros(Nx, N+1);
init_state(:,1) = x0;
for idx = 2:N+1
    init_state(:,idx) = init_state(:,idx-1) + dt * sys_h_2(init_state(:,idx-1),
init_control(:,min(idx-1,N)), prob_info);
end
eta0 = [reshape(init_state, [], 1); reshape(init_control, [], 1)];

% Solve optimization
% Output the optimal solution eta_star, from which the state x_star and control u_star can be
extracted
fun      = @(eta) cost_2(eta, prob_info);
nonlcon = @(eta) nconst_2(eta, prob_info);

[eta_star, cost_star, exitflag, output] = fmincon(fun, eta0, [], [], [], [], lb, ub, nonlcon,
options);

% A mechanism to detect whether the current optimization is successful. If the optimization
fails (for example,
% the state or input does not satisfy the constraints), the entire RHP cycle is aborted.
fprintf("Exit flag: %d\n", exitflag);
disp(output.message)

if exitflag <= 0
    warning('fmincon failed. Exiting loop early.');
```

```

    break;
end

% Parse solution
x_star = reshape(eta_star(1:(N+1)*Nx), Nx, N+1);
u_star = reshape(eta_star((N+1)*Nx+1:end), Nu, N);

% Only save the trajectory and control results of the first half
% Re-optimize the next paragraph
K = round(N/2);
x_all = [x_all, x_star(:,2:K+1)];
u_all = [u_all, u_star(:,1:K)];
t_all = [t_all, t0 + (1:K)*dt];

% Update state
x0 = x_star(:,K+1);
t0 = t0 + K*dt;

```

```

end

total_time = toc;
fprintf("Total RHP planning time: %.2f seconds\n", total_time)

% === Final Visualization ===

figure('Name','RHP Trajectory Summary','Position',[100 100 1200 800])

subplot(2,2,1)
% Set up a good color
plot(x_all(1,:), x_all(2,:), 'Color', [0.4 0.6 1], 'LineWidth', 2);
hold on
plot(x_ref_full, y_ref_full, 'r--')
viscircles(prob_info_fixed.centres, prob_info_fixed.radii *
ones(size(prob_info_fixed.centres,1),1));
legend('Optimized Trajectory', 'Reference Trajectory')
axis equal, xlabel('x (m)'), ylabel('y (m)'), title('Trajectory Tracking')

grid on

subplot(2,2,2)
plot(rad2deg(x_all(3,:)))
xlabel('Step'), ylabel('Heading (deg)'), title('Vehicle Heading')
grid on

subplot(2,2,3)
plot(x_all(4,:))
xlabel('Step'), ylabel('Speed (m/s)'), title('Forward Speed')
grid on

subplot(2,2,4)
plot(u_all(1,:), 'b')
hold on
plot(u_all(2,:), 'r')
legend('Steering Input', 'Acceleration')
xlabel('Step'), title('Control Inputs')
grid on

% ===== Complete RHP Trajectory over Parkinglot Map, Obstacles, Must-pass
% Points,Reference Trajectory =====
figure('Name', 'Complete RHP Visualization', 'Position', [200, 200, 1000, 800]);
hold on;
axis equal;
xlabel('X (meters)');
ylabel('Y (meters)');

```

```

title('Binary Occupancy Grid and Trajectories');
grid on;

% Re-built binaryOccupancyMap
mapWidth = 25;
mapHeight = 25;
resolution = 10;
local_map = binaryOccupancyMap(mapWidth, mapHeight, resolution);

% Draw the stop line (black line)
for y = [6, 10, 14, 18]
    setOccupancy(local_map, [1:0.1:6; y*ones(1, length(1:0.1:6))]', 1);
end
setOccupancy(local_map, [1*ones(1, length(6:0.1:22)); 6:0.1:22]', 1);
setOccupancy(local_map, [1:0.1:6; 22*ones(1, length(1:0.1:6))]', 1);

for y = [6, 10, 14, 18]
    setOccupancy(local_map, [12:0.1:17; y*ones(1, length(12:0.1:17))]', 1);
end
setOccupancy(local_map, [17*ones(1, length(6:0.1:22)); 6:0.1:22]', 1);
setOccupancy(local_map, [12:0.1:17; 22*ones(1, length(12:0.1:17))]', 1);

% Draw the obstacle vehicle (black square)
markRectangle(local_map, 2, 7.1, 2.4, 1.8);
markRectangle(local_map, 2, 11.1, 2.4, 1.8);
markRectangle(local_map, 2, 19.1, 2.4, 1.8);
markRectangle(local_map, 13, 7.1, 2.4, 1.8);
markRectangle(local_map, 13, 19.1, 2.4, 1.8);
markRectangle(local_map, 8.1, 7.8, 1.8, 2.4);
markRectangle(local_map, 0, 2.1, 2.4, 1.8);

% Inflate map
inflate(local_map, 1.1);

% Show parking lot background
show(local_map);

% Draw obstacles (red circles)
theta = linspace(0, 2*pi, 100);
r = 3;
obstacles = [3.2 14.0; 3.2 20.0; 14.2 8.0; 14.2 20.0];
for i = 1:size(obstacles,1)
    plot(obstacles(i,1) + r*cos(theta), obstacles(i,2) + r*sin(theta), 'r-', 'LineWidth', 2);
end

% Must-pass waypoints (black crosses)

```

```

wp = [0.2 20 20 8 8 13.2; 3 3 22 22 16 16; 0 pi/2 pi 3*pi/2 2*pi 2*pi];
plot(wp(1,:), wp(2,:), 'kx', 'MarkerSize', 10, 'LineWidth', 2);

% Reference Trajectory from Task 2.1 (red dashed line)
[x_ref_full, y_ref_full, ~, ~, ~, ~, ~, ~, ~] = Task2_2_ref(0.1);
plot(x_ref_full, y_ref_full, 'r--', 'LineWidth', 1.5);

% Optimized RHP trajectory (blue solid line)
plot(x_all(1,:), x_all(2,:), 'Color', [0.4 0.6 1], 'LineWidth', 2);

% Local function to mark rectangles
function markRectangle(map, x, y, w, h)
    xs = x : 0.05 : (x + w);
    ys = y : 0.05 : (y + h);
    [X, Y] = meshgrid(xs, ys);
    setOccupancy(map, [X(:), Y(:)], 1);
end

% Manually create a proper legend handles
h1 = plot(NaN, NaN, 'r-', 'LineWidth', 2); % Obstacles
h2 = plot(NaN, NaN, 'kx', 'MarkerSize', 10, 'LineWidth', 2); % Must-pass waypoints
h3 = plot(NaN, NaN, 'r--', 'LineWidth', 1.5); % Reference trajectory
h4 = plot(NaN, NaN, 'Color', [0.4 0.6 1], 'LineWidth', 2); % Optimized trajectory

legend([h1 h2 h3 h4], {'Obstacles', 'Must-pass Waypoints', 'Reference Trajectory', 'Optimized Trajectory'}, 'Location', 'bestoutside');

```


Appendix 2: Reference

- [1] The MathWorks, Inc., “fmincon documentation – Constrained nonlinear minimization,” MathWorks Documentation, 2023. [Online]. Available: <https://www.mathworks.com/help/optim/ug/fmincon.html>
- [2] MathWorks, “Planning and Control - MATLAB & Simulink,” 2023
- [3] MathWorks, “Path Planning and Following - MATLAB & Simulink,” 2023
- [4] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Nob Hill Publishing, 2017.